

UNIT - III (COMPLETE)

Basic MongoDB Operations

End Semester Examination Notes | SRM University Delhi-NCR | 23CAF3010 NOSQL & MongoDB

COMPLETE SYLLABUS COVERAGE	Topics Included
1. Data Types in Mongo Shell (18 types)	String, Integer, Double, Boolean, Null, Array, Object, ObjectId, Undefined, Binary, Date, MinKey/MaxKey, Symbol, Regex, JavaScript, Timestamp, Decimal
2. Inserting & Saving Documents	insertOne(), insertMany(), insert(), save() — syntax + examples
3. Batch Insert	Bulk.insert(), Unordered & Ordered bulk operations
4. Removing Documents	deleteOne(), deleteMany() — filter examples
5. Updating Documents	updateOne(), updateMany(), \$set, \$unset, \$push, \$inc, \$mul
6. Update Top Level Field	\$dollar prefixed fields, \$setField, aggregation pipeline update
7. Update an Embedded Field	Dot notation with \$set, nested document updates
8. Update Multiple Documents	updateMany() with filter conditions
9. Replace a Document	replaceOne() — full document replacement
10. MongoDB Commands	adminCommand, aggregate, cloneDatabase, createCollection, createView, dropDatabase, getLogComponents
11. Limitations in Querying Data	16MB size limit, index limit, cursor.limit(), pagination
12. Query for All Documents	find({}), SELECT * equivalent
13. Query by a Top Level Field	find({field:value}), comparison operators, \$and, \$or
14. Aggregation	\$match, \$group, \$sort, \$project, \$limit, \$skip, \$unwind, \$lookup
15. Indexing	createIndex(), types, explain(), performance benefits
16. Replica Sets	Primary/Secondary/Arbiter, elections, oplog, high availability
END SEM 16M QA Practice	5 full question-answer sets from MST-II paper

SECTION 1: Data Types in Mongo Shell

In MongoDB, documents are stored in BSON (Binary Encoded Serialization Of JSON) which is the binary encoded format of JSON. BSON supports various data types beyond those available in standard JSON. MongoDB drivers provide native support for these data types in host languages, and the Mongo Shell provides several helper classes to support their use in Mongo JavaScript Shell.

Key points about BSON:

- BSON is a superset of JSON — it supports all JSON types PLUS additional types like Date, ObjectId, Binary Data, etc.
 - We can make Remote Procedural Calls (RPC) in MongoDB using BSON format.
 - MongoDB BSON provides support for additional data types than standard JSON.
-

1.1 String

This is the most commonly used data type in MongoDB to store data. BSON strings are of UTF-8 encoding. So, the drivers for each programming language convert from the string format of the language to UTF-8 while serializing and de-serializing BSON. The string must be a valid UTF-8 string.

```
// Insert multiple students with string names:
db.student.insertMany([ { name: "Akshay" }, { name: "Vikash" } ])

// Insert a single document with string field:
db.student.insertOne( { name: "Akash", age: 18 } )
```

1.2 Integer

In MongoDB, the integer data type is used to store an integer (whole number) value. We can store integer data type in two forms: 32-bit signed integer and 64-bit signed integer. The type to use depends on the driver and size of the number.

```
// 32-bit integer (age field):
db.student.insertOne( { name: "Akash", age: 18 } )

// 64-bit integer (large numbers):
db.logs.insertOne( { eventCount: NumberLong("9876543210") } )
```

1.3 Double

The double data type is used to store floating-point values. It is used whenever a decimal number is needed. This maps to the IEEE 754 double-precision floating-point format.

```
// Store a double (marks with decimal):
db.student.insertOne( { name: "Sagar", age: 20, marks: 546.63 } )
```

1.4 Boolean

The Boolean data type is used to store either true or false. It is used to represent binary states such as pass/fail, active/inactive, etc.

```
// Boolean field pass:
db.student.insertOne( { name: "Vishal", pass: true } )
```

1.5 Null

The null data type is used to store the null value. It represents a field that exists but has no value assigned to it. Note: null is different from a field not existing at all.

```
// MobNo field is null (exists but empty):
db.student.insertOne({ name: "Akash", MobNo: null, skills: ["C", "C++", "Java", "Python", "JS"] })
```

1.6 Array

The Array is a set of values. It can store the same or different data type values in it. In MongoDB, arrays are created using square brackets []. Arrays are extremely useful for storing multiple values in a single field — for example, a list of skills or tags.

```
// Student with skills array:
db.student.insertOne({ name: "Akash", skills: ["C", "C++", "Java", "Python", "JS"] })

// Mixed type array:
db.sample.insertOne({ data: [1, "hello", true, null, 3.14] })
```

1.7 Object (Embedded Document)

Object data type stores embedded documents (also called nested or subdocuments). Embedded documents are those types of documents which contain a document inside another document. This allows hierarchical data storage — for example, an address embedded inside an employee document.

```
// Embedded document:
db.book.insertOne({ Book: { name: "C in depth", writer: "Akash" } })

// Employee with embedded address:
db.employees.insertOne({
  name: "Tamanna",
  salary: 50000,
  address: { city: "Delhi", pincode: 110001 }
})
```

★ **Exam Tip:** *Embedded fields are accessed using dot notation: "address.city". This is heavily tested in queries and updates.*

1.8 ObjectId

Whenever we create a new document in the collection, MongoDB automatically creates a unique ObjectId for that document. Every document has an `_id` field. The data stored in `_id` is in hexadecimal format. The length of the ObjectId is exactly 12 bytes which consist of:

- 4 bytes → Timestamp value (seconds since Unix epoch)
- 3 bytes → Machine identifier (machine ID)
- 2 bytes → Process ID
- 3 bytes → Random counter (incremented for each new ObjectId)

```
// Retrieve and view ObjectId:
db.book.find().pretty()

// Output:
{ "_id": ObjectId("6daf7166d54aa34c96dfg1"), "Book": { "name": "C in depth", "writer": "Akash" } }

// Access the ObjectId of a new insert:
var result = db.students.insertOne({ name: "Raj" })
print(result.insertedId)
```

1.9 Undefined

This data type stores the undefined values. Although it is a valid BSON type, it is deprecated in the BSON spec. In the Mongo Shell, undefined behaves differently from null. null means "no value", whereas undefined means "value not assigned".

```
db.project.insertOne({ name: "Paul", duration: undefined, binaryValue: "11001100" })
```

1.10 Binary Data

This data type is used to store binary data — data that is not representable as text. Binary data is useful for storing images, files, encrypted content, and other raw bytes. It is stored as a BSON BinData type.

```
// Storing binary value (as string representation):
db.project.insertOne({ name: "Paul", duration: undefined, binaryValue: "11001100" })
```

1.11 Date

Date data type stores dates. Internally, it is a 64-bit integer which represents the number of milliseconds since the Unix epoch (January 1, 1970). BSON data type generally supports UTC datetime and it is signed. If the value of the date data type is negative, it represents the dates before 1970. There are various methods to return date:

- `Date()` — returns the current date in STRING format.
- `new Date()` — returns a date OBJECT, uses the `ISODate()` wrapper.
- `new ISODate()` — also returns a date object, uses the `ISODate()` wrapper.

```
var date1 = Date()      // string: 'Tue May 12 2026 ...'
var date2 = new Date()  // ISODate object
var date3 = new ISODate() // ISODate object
db.date.insertOne({ Date: date1, Date2: date2, Date3: date3 })
```

1.12 Min Key and Max Key

Min Key and Max Key are special internal BSON data types used for comparison purposes. Min Key compares the value of the lowest BSON element. Max Key compares the value against the highest BSON element. Both are internal data types — they are used internally by MongoDB for range queries and sorting.

```
db.m.insertOne({ min: MinKey, max: MaxKey })
```

1.13 Symbol

This data type is similar to the string data type. It is generally NOT supported by Mongo Shell, but if the Shell gets a symbol from the database, it converts this type into a string type automatically. It is essentially deprecated in modern MongoDB usage.

```
var symb1 = "nik432#"
db.symb1.insert({ symbol: symb1 })
WriteResult({ "nInserted": 1 })
```

1.14 Regular Expression

This data type is used to store regular expressions directly in MongoDB documents. Regular expressions can be used in queries to search for pattern matches in string fields.

```
var expression = new RegExp("^gfg")
db.exp.insertOne({ Regular_Expression: expression })

// Query using regex:
db.students.find({ name: /^Ak/ }) // names starting with Ak
```

1.15 JavaScript

This data type is used to store JavaScript code into the document without the scope. The JavaScript code can be executed within the MongoDB JavaScript engine.

```
db.js.insertOne({ code: function() { return 'hello'; } })
```

1.16 JavaScript with Scope

This MongoDB data type stores JavaScript data with a scope. A scope allows the JavaScript function to have access to external variables. JavaScript with scope = JavaScript function + a scope document (mapping of variable names to values).

```
var date1 = Date()
var date2 = new Date()
var date3 = new ISODate()
db.date.insertOne({ Date: date1, Date2: date2, Date3: date3 })
```

1.17 Timestamp

In MongoDB, this data type is used to store a timestamp. It is useful when we modify our data to keep a record of when changes occurred. The value of this data type is 64-bit. Timestamps are used internally by MongoDB for the oplog (operations log) in replication.

```
var timestamp = new Timestamp()
db.time.insertOne({ Timestamp: timestamp })
```

1.18 Decimal (NumberDecimal)

This MongoDB data type stores a 128-bit decimal based floating-point value. It provides much higher precision than the standard double type and is used for financial calculations and scientific computations that require exact decimal representation.

```
var decimal = NumberDecimal("1000.55")
db.decimal.insertOne({ NumberDecimal: decimal })

// Used when exact precision is required (e.g., currency):
db.accounts.insertOne({ balance: NumberDecimal("9876543210.99") })
```

1.19 Summary Table — All Data Types

Data Type	Description	Example Value / Syntax
String	Most common; must be valid UTF-8	"Akash", "Delhi"
Integer	32-bit or 64-bit signed whole number	18, NumberLong('9876543210')
Double	64-bit floating-point decimal	546.63, 3.14
Boolean	true or false only	true, false
Null	Null/absent value	null
Array	Set of values in square brackets	["C", "Java", "Python"]
Object	Embedded/nested document	{ city: "Delhi", pin: 110001 }
ObjectId	12-byte auto-generated unique _id	ObjectId('...')
Undefined	Deprecated; undefined value	undefined
Binary Data	Raw binary/byte data	BinData(0,"11001100")
Date	64-bit ms since Unix epoch, UTC	new Date(), Date()

Min/Max Key	BSON internal comparison keys	MinKey, MaxKey
Symbol	Like String; auto-converts in Shell	"nik432#"
Regex	Regular expression pattern	new RegExp("^gfg")
JavaScript	JS code without scope	function() { }
JS with Scope	JS code with variable scope	Code + scope document
Timestamp	64-bit; used in oplog/replication	new Timestamp()
Decimal128	128-bit high-precision decimal	NumberDecimal("1000.55")

★ **Exam Tip:** *ObjectId = 12 bytes: 4(timestamp) + 3(machine) + 2(process) + 3(counter). This is a direct MCQ question. Also: Date() returns STRING, new Date() returns OBJECT.*

SECTION 2: Inserting and Saving Documents

The MongoDB Shell provides the following methods to insert documents into a collection. MongoDB stores data as BSON documents. When you insert a document, if it does not specify an `_id` field, MongoDB automatically adds an `_id` field with an `ObjectId` value.

2.1 `insertOne()` — Insert a Single Document

`db.collection.insertOne()` inserts a single document into a collection. If the document does not specify an `_id` field, MongoDB adds the `_id` field with an `ObjectId` value to the new document. `insertOne()` returns a document that includes the newly inserted document's `_id` field value.

Syntax:

```
db.collectionName.insertOne(document)
```

Example 1 — Basic `insertOne()`:

```
db.createCollection('EmpDetails')
db.EmpDetails.insertOne({
  firstname: "Radhika",
  lastname: "Sharma",
  DateOfBirth: "1995-09-26",
  email: "radhika_sharma.123@gmail.com",
  phone: "9848022338"
})
```

Example 2 — Movie document with array and nested fields:

```
db.movies.insertOne({
  title: "The Favourite",
  genres: ["Drama", "History"],
  runtime: 121,
  rated: "R",
  year: 2018,
  directors: ["Yorgos Lanthimos"],
  cast: ["Olivia Colman", "Emma Stone", "Rachel Weisz"],
  type: "movie"
})
```

 **Note:** `insertOne()` returns: `{ acknowledged: true, insertedId: ObjectId('...') }`

2.2 `insertMany()` — Insert Multiple Documents

`db.collection.insertMany()` can insert multiple documents into a collection in a single operation. Pass an array of documents. By default, `insertMany()` performs an unordered insert — if one document fails, MongoDB continues inserting the rest. Use `ordered: true` to stop on first error.

Syntax:

```
db.collectionName.insertMany([doc1, doc2, doc3, ...])
```

Example:

```
db.movies.insertMany([
  {
    title: "Jurassic World: Fallen Kingdom",
    genres: ["Action", "Sci-Fi"],
    runtime: 130,
    rated: "PG-13",
    year: 2016,
    directors: ["J. S. A-Bayona"],
    type: "movie"
  },
  {
    title: "The Favourite",
    genres: ["Drama", "History"],
    runtime: 121,
    rated: "R",
    year: 2018,
    directors: ["Yorgos Lanthimos"],
    cast: ["Olivia Colman", "Emma Stone", "Rachel Weisz"],
    type: "movie"
  }
])
```

```
{
  title: "Tag",
  genres: ["Comedy", "Action"],
  runtime: 105,
  rated: "R",
  year: 2018,
  directors: ["Jeff Tomsic"],
  type: "movie"
}
```

★ **Exam Tip:** *MST Q1: Which statement about insertMany() is correct? Answer: c) Can insert multiple documents in a single operation. It does NOT stop if one document fails by default.*

2.3 insert() Method (Legacy)

To insert data into a MongoDB collection, you can also use MongoDB's older insert() method. It can insert a single document or an array of documents. This method is deprecated in newer MongoDB versions but still appears in exams.

Syntax:

```
db.collectionName.insert(document)
```

Example:

```
db.users.insert({
  _id: ObjectId("507f191e810c19729de860ea"),
  title: "MongoDB overview",
  description: "MongoDB is a NoSQL database",
  tags: ["mongodb", "database", "NoSQL"],
  likes: 100
})
WriteResult({ "nInserted": 1 })
```

2.4 save() Method

db.collection.save() updates an existing document or inserts a new document depending on its document parameter. Its behaviour depends on whether the document has an _id field:

- **INSERT** (no _id field): If the document does not contain an _id field, the save() method calls the insert method. A new document is created.
- **UPDATE** (with _id field): If the document contains an _id field, the save() method is equivalent to an update with the upsert option set to true. It performs a complete replacement of the matched document.
- **REPLACE**: The save() method performs an update with upsert: true since the document contains an _id field.

Syntax:

```
db.collectionName.save(<document>, { writeConcern: <document> })
```

Examples:

```
// INSERT — no _id, so save() behaves like insert():
db.products.save({ item: 'Book', qty: 40 })

// REPLACE — has _id, so save() behaves like update with upsert:
db.product.save({ _id: 100, item: 'water', qty: 30 })

// REPLACE mode — The save() method performs update with upsert:true
// Since doc contains _id field. If _id:100 exists → replace it.
// If not exists → insert it.
```


 **Note:** `save()` with `_id` performs the update with `upsert:true` (replaces entire document if exists, inserts if not). It does NOT use `$set` — it replaces the whole document.

★ **Exam Tip:** True/False Q5: `replaceOne()` keeps existing fields NOT mentioned in the new document. Answer: FALSE. `replaceOne()` and `save()` (with `_id`) REMOVE all old fields.

SECTION 3: Batch Insert

Batch Insert (also called Bulk Write) allows you to perform multiple insert, update, or delete operations in a single database call. This dramatically reduces the number of round-trips to the server and improves performance when working with large datasets.

The Bulk API provides:

- `Bulk.insert()` — adds an insert to a bulk operation list
- `bulk.execute()` — executes all operations in the list in one call

There are 2 types of Bulk Insertion:

3.1 Unordered Insertion of Documents

In an unordered bulk insert (`db.collection.initializeUnorderedBulkOp()`), if one document fails (e.g., duplicate key error), MongoDB CONTINUES inserting the remaining documents. The operations may be executed in any order by MongoDB for performance optimization.

When to use: When you want maximum throughput and don't need strict ordering.

```
var bulk = db.students.initializeUnorderedBulkOp();
bulk.insert({ firstname: "Sachin", lastname: "Tendulkar" });
bulk.insert({ firstname: "Virender", lastname: "Sehwag" });
bulk.execute();
```

3.2 Ordered Insertion of Documents

In an ordered bulk insert (`db.collection.initializeOrderedBulkOp()`), if one document fails, MongoDB STOPS processing the remaining documents immediately. Operations are executed in the exact order they were added. This guarantees sequential execution but is slower.

When to use: When order matters and you need to stop on first error.

```
var bulk = db.students.initializeOrderedBulkOp();
bulk.insert({ firstname: "Robin", lastname: "Manvin" });
bulk.insert({ firstname: "John", lastname: "Hudson" });
bulk.execute();
```

3.3 Bulk Write Methods Available

Bulk Write Method	Operation Performed
<code>insertOne</code>	Inserts a single document into the collection
<code>updateOne</code>	Updates the first document matching the filter
<code>updateMany</code>	Updates all documents matching the filter
<code>replaceOne</code>	Replaces the first document matching the filter
<code>deleteOne</code>	Deletes the first document matching the filter
<code>deleteMany</code>	Deletes all documents matching the filter

3.4 Batch Insert Using `insertMany` (Direct Method)

The simplest batch insert — no need for Bulk API when just inserting:

```
db.pizzas.insertMany([
  { _id: 0, type: "Pepperoni", size: "Small", price: 42 },
  { _id: 1, type: "Cheese", size: "Medium", price: 73 }
]);
```

3.5 Bulk.insert() — The Formal Batch Insert

For precise bulk control with `execute()`:

```
// Using formal Bulk API:
var bulk = db.items.initializeUnorderedBulkOp();
bulk.insert({ item: "abc123", defaultQty: 100, status: "A", points: 100 });
bulk.insert({ item: "ijk123", defaultQty: 200, status: "A", points: 200 });
bulk.execute();
```

Unordered Bulk Insert	Ordered Bulk Insert
<code>initializeUnorderedBulkOp()</code>	<code>initializeOrderedBulkOp()</code>
Continues on error	Stops on first error
Faster (parallel-friendly)	Slower (sequential)
Order of execution not guaranteed	Strict sequential order
Use for max performance	Use when order matters

★ **Exam Tip:** *Batch Insert has 2 types: Unordered (continues on error) and Ordered (stops on error). The `bulk.execute()` must be called to actually perform the operations.*

SECTION 4: Removing / Deleting Documents

MongoDB provides two main methods for deleting documents from a collection. Both methods accept a filter document (query condition) to specify which documents to remove.

4.1 deleteMany() — Delete Multiple Documents

Deletes all documents that match the specified filter. If you pass an empty filter {}, it deletes ALL documents in the collection (but the collection itself remains).

Syntax:

```
db.collectionName.deleteMany(filter)
```

Examples:

```
// Delete ALL documents in the movies collection:
db.movies.deleteMany({})

// Delete all movies titled 'Titanic':
db.movies.deleteMany({ title: "Titanic" })

// Delete all products where stock = 0:
db.products.deleteMany({ stock: 0 })

// Delete all students with marks < 33:
db.students.deleteMany({ marks: { $lt: 33 } })
```

4.2 deleteOne() — Delete a Single Document

Deletes the FIRST document that matches the filter. Even if multiple documents match, only one is deleted. The document deleted is the first one found by the collection scan order.

Syntax:

```
db.collectionName.deleteOne(filter)
```

Examples:

```
// Delete the first movie with Brad Pitt in the cast:
db.movies.deleteOne({ cast: "Brad Pitt" })

// Delete employee named Tamanna:
db.employees.deleteOne({ name: "Tamanna" })
```

4.3 Comparison: deleteOne vs deleteMany

deleteOne()	deleteMany()
Deletes FIRST matching document only	Deletes ALL matching documents
Safe for targeted removal	Use with careful filter to avoid data loss
Returns deletedCount: 1 if found	Returns deletedCount: n (number removed)
db.col.deleteOne({ name: "A" })	db.col.deleteMany({ status: "inactive" })

★ **Exam Tip:** *deleteMany({}) with EMPTY filter deletes ALL documents. The collection still exists — only the documents are removed.*

SECTION 5: Updating Documents

MongoDB provides update operators to modify field values without replacing the entire document. The main update methods are `updateOne()`, `updateMany()`, and `replaceOne()`. MongoDB also provides useful operators like `$set`, `$unset`, `$push`, `$pull`, `$inc`, `$mul` for targeted modifications.

5.1 Common Update Operators

Update Operator	Description & Usage
<code>\$set</code>	Sets the value of a field. Creates it if it doesn't exist.
<code>\$unset</code>	Removes a specified field from a document.
<code>\$inc</code>	Increments a field value by a specified amount.
<code>\$mul</code>	Multiplies a field value by a specified number (e.g., $\text{price} * 1.10 = 10\% \text{ increase}$).
<code>\$push</code>	Adds an element to an array field.
<code>\$pull</code>	Removes matching elements from an array.
<code>\$addToSet</code>	Adds to array only if element does not already exist.
<code>\$rename</code>	Renames a field.
<code>\$currentDate</code>	Sets a field to the current date as a Date or Timestamp.
<code>\$min</code>	Updates field only if new value is LESS than current value.
<code>\$max</code>	Updates field only if new value is GREATER than current value.

★ **Exam Tip:** *MST Q3: What happens if `$set` is used on a non-existing field? Answer: c) Field is CREATED. `$set` creates the field if it doesn't exist.*

5.2 `updateOne()` — Update a Single Document

Updates the FIRST document that matches the filter. Uses update operators (like `$set`) to change specific fields. Only the specified fields are modified — all other fields remain untouched.

Syntax:

```
db.collection.updateOne(filter, update, options)
```

Example 1 — Basic `updateOne` with `$set` and `$currentDate`:

```
await db.collection('inventory').updateOne(
  { item: 'Paper' },      // filter: find doc where item='Paper'
  {
    $set: {               // $set: update these specific fields
      'size.uom': 'cm',   // update nested field using dot notation
      status: 'P'
    },
    $currentDate: {       // set lastModified to current date
      lastModified: true
    }
  }
);
```

Example 2 — Adding a new status field to a student:

```
db.students.updateOne(
  { name: "Akash" },
  { $set: { status: "Active" } }
)
```

5.3 updateMany() — Update Multiple Documents

Updates ALL documents that match the filter condition. This is the method to use when you want to apply a change to many records at once.

Syntax:

```
db.collection.updateMany(filter, update, options)
```

Example 1 — Update all inventory items with qty < 50:

```
await db.collection('inventory').updateMany(
  { qty: { $lt: 50 } },      // filter: all docs where qty < 50
  {
    $set: { 'size.uom': 'in', status: 'P' },
    $currentDate: { lastModified: true }
  }
);
```

Example 2 — Increase price of all 'Electronics' products by 10%:

```
db.products.updateMany(
  { category: "Electronics" },
  { $mul: { price: 1.10 } } // multiply price by 1.10 = 10% increase
)
```

Example 3 — Add status 'Topper' to students who have Python skill and marks > 80:

```
db.students.updateMany(
  {
    skills: "Python",
    marks: { $gt: 80 }
  },
  { $set: { status: "Topper" } }
)
```

5.4 Update Top Level Field

MongoDB field names can be: (1) Dollar-prefixed (\$price), or (2) Containing periods (unit.price). These are NOT directly accessible via standard update operators. Instead, helper functions are used:

- \$setField — sets a field by name (can handle \$ and . in field names)
- \$getField — retrieves a field by name
- \$literal — treats a string literally (without interpreting \$ as operator)

Key rule: At top level → \$ prefixed names treated as operators. Inside stored data fields → sub-field names are fine.

```
// Insert with dollar-prefixed field (allowed):
db.sales.insertOne({ "$price": "50-00", "quantity": 30 })

// Nested level dollar-prefixed (allowed):
db.book.insertOne({ name: "C in depth", $id: 123, location: { "$db": "novels" } })

// Update using aggregation pipeline + $setField:
db.school.aggregate([
  { $match: { "registered": true } },
  { $replaceWith: {
    $setField: {
```

```
    field: { $literal: "$term" },
    input: "$$ROOT",
    value: "Spring"
  }
} }
```

 **Note:** MongoDB warns about \$ fields. Fields starting with \$ can cause confusion with operators in update, aggregation, or replacement operations.

5.5 Update an Embedded (Nested) Field

To update a field inside a nested/embedded document, use DOT NOTATION inside \$set. MongoDB will update only that specific nested field without affecting the parent document or its other fields.

```
// Update the 'street' inside 'address' embedded document:
db.housing.updateOne(
  { "_id": "E123" },
  { $set: { "address.$street": "elm mews" } }
)

// Update city inside address for employees in Delhi:
db.employees.updateMany(
  { "address.city": "Delhi" },
  { $set: { "address.city": "Mumbai" } }
)
```

 **Note:** You CAN update address.\$street (nested field value using dot notation). You CANNOT update \$rooms as it is a \$ prefixed TOP-level field name — those require \$setField.

★ **Exam Tip:** MST Q2: Which operator is used to update a nested (embedded) field? Answer: c) Dot notation with \$set. Use 'address.city' inside \$set to update nested field.

SECTION 6: Replace a Document — replaceOne()

replaceOne() replaces the ENTIRE content of a matched document with a completely new document. The _id field is preserved (it cannot be replaced). All other original fields are REMOVED and replaced with the new document's fields. The replacement document CANNOT contain update operators like \$set.

Syntax:

```
db.collection.replaceOne(filter, replacementDocument, options)
```

Example 1 — Replace an inventory document completely:

```
await db.collection('inventory').replaceOne(
  { item: 'Paper' },    // find document where item='Paper'
  {                    // completely replace with this new document
    item: 'Paper',
    inStock: [
      { warehouse: 'A', qty: 60 },
      { warehouse: 'B', qty: 40 }
    ]
  }
);
```

Example 2 — From MST-II paper (Replace one employee completely):

```
db.employees.replaceOne(
  { name: "Tamanna" }, // find Tamanna
  {                    // replace ENTIRE document
    name: "Sanjeevni",
    salary: 60000,
    address: {
      city: "Pune",
      pincode: 411001
    }
  }
);
```

Critical difference between replaceOne() and updateOne() with \$set:

replaceOne()	updateOne() with \$set
REPLACES the entire document	Updates ONLY the specified fields
All old fields NOT in new doc are DELETED	All old fields not in \$set are KEPT
Does not use update operators (\$set etc.)	Must use update operators (\$set, \$inc etc.)
_id field is preserved	_id field is preserved
Use when: complete document overwrite needed	Use when: only some fields need changing
replaceOne({name:"A"}, {name:"B", age:25})	updateOne({name:"A"}, {\$set:{age:25}})

★ **Exam Tip:** True/False Q5: replaceOne() keeps existing fields not mentioned in new document. Answer: FALSE. replaceOne REMOVES all old fields and replaces with the new document.

SECTION 7: MongoDB Database Commands

MongoDB provides various database-level commands accessible via the Mongo Shell. These are administrative and operational commands that work at the database level (not collection level).

7.1 db.adminCommand(cmd)

The admin command method runs against the admin database to run specified database commands by providing a helper. The command argument is specified in document form or string form. If the command is defined as a string, it cannot include any argument.

```
db.adminCommand({
  createUser: "SukmPoint",
  pwd: PasswordPrompt(),
  roles: [
    { role: "dbowner", db: "admin" }
  ]
})
```

7.2 db.aggregate([pipeline], options)

The aggregate method initializes a specific diagnostic or admin pipeline which does not require any underlying collection. The pipeline parameter always starts with a compatible stage such as \$listLocalSessions. It is an array of stages that will be executed.

```
db.aggregate([
  { $currentOp: { allUsers: true, idleConnections: true } },
  { $match: { shard: "ShardDemo" } }
])
```

7.3 db.cloneDatabase('hostname')

The cloneDatabase method copies the specified database to the current database and assumes that the database at the remote location has the same name as the current database.

```
db.cloneDatabase("customers")
```

7.4 db.commandHelp(command)

We have the help option for the specified database command using the commandHelp method. The command parameter contains the name of the database command.

```
db.commandHelp("find")
```

7.5 db.createCollection(name, options)

A new collection or view will be created using this method. The createCollection method is used primarily for creating new collections that use specific options (like capped collections, validators) when the collection is first referenced in a command. Normally MongoDB creates collections automatically on first insert.

```
// Simple collection creation:
db.createCollection("students")

// Capped collection (fixed size, auto-overwrites oldest when full):
db.createCollection("logs", { capped: true, size: 5242880, max: 5000 })
```

7.6 db.createView()

When applying the specified aggregation pipeline to the collection, the `createView` method creates a new view for the collection. Views act as read-only virtual collections. They can be computed during read operations and act as read-only. The views can be created in the same database of the source collection.

```
db.createView(<view>, <source>, <pipeline>, <options>)  
  
// Example — create a view of students scoring above 75:  
db.createView(  
  "topStudents",    // view name  
  "students",       // source collection  
  [ { $match: { marks: { $gt: 75 } } } ] // pipeline  
)
```

7.7 db.dropDatabase(writeConcern)

The `drop` method removes the specified database entirely and deletes all associated data files. This is irreversible. Use with extreme caution.

```
use temp  
db.dropDatabase()
```

7.8 db.getLogComponents()

The `getLogComponents` method returns the current log verbosity settings. The method determines the amount of log messages produced by MongoDB for each log message component (query, replication, storage, etc.).

```
db.getLogComponents()
```

SECTION 8: Limitations in Querying Data

MongoDB has several important limitations that affect how data can be stored and queried. Understanding these limitations is important for designing efficient MongoDB applications.

8.1 Document Size Limit

MongoDB has a maximum document size of 16 MB (16 megabytes). If a document exceeds this limit, it CANNOT be stored in MongoDB. For larger binary data (like images, videos), use MongoDB's GridFS which splits the file into chunks.

16MB Limit

Maximum size for a single BSON document. Documents larger than 16MB cannot be stored. Use GridFS for large files.

8.2 Index Limit

MongoDB imposes a limit on the number of indexes that can be created for a single collection. The maximum is 64 indexes per collection. Also, the index key size limit is 1024 bytes (in older versions). Compound indexes can have at most 32 fields.

8.3 Cursor Limit — limit() Method

The limit in MongoDB is used to limit the number of documents returned by a query. It takes a single argument that specifies the maximum number of documents to return. The limit in MongoDB can be used with the find() method or the aggregate() method.

Syntax:

```
Cursor.limit(n)
db.collection.find().limit(10)
```

Key points about limit():

1. The limit in MongoDB can be used with find() or aggregate() queries.
2. The limit parameter must be a POSITIVE integer.
3. If limit is not specified, MongoDB returns ALL matching documents.
4. If query returns fewer documents than the limit, the limit has no effect.
5. limit() can be chained with skip() to implement pagination.

Example — Limit and Sort:

```
// Return first 10 documents sorted by name ascending:
db.students.find().sort({ name: 1 }).limit(10)

// Pagination: page 2 with 10 items per page:
db.students.find().sort({ name: 1 }).skip(10).limit(10)
```

 **Note:** *Cursor.limit(0) means NO limit — MongoDB returns all documents.*

8.4 How limit() Works Internally

When you call limit() on a MongoDB query, the MongoDB Server stops returning results once it has reached the specified limit. The limit works by adding a limit parameter to the query — when the query is executed on the MongoDB Server, it will stop returning results once it has reached the specified limit.

```
// Using limit with find():
db.collection.find().limit(10)

// Using limit with aggregate():
```

```
db.collection.aggregate([ { $limit: 5 } ])
```

SECTION 9: Querying Documents

MongoDB provides powerful query capabilities using the `find()` method. The `find()` method accepts a filter document (query condition) and optionally a projection document (which fields to return).

Syntax:

```
db.collection.find(filter, projection)
```

9.1 Query for All Documents in a Collection

To select all documents in the collection, pass an empty document as the query filter parameter to the `find()` method:

```
db.inventory.find({})

// With pretty printing:
db.inventory.find({}).pretty()

// Equivalent SQL:
// SELECT * FROM inventory
```

9.2 Query by a Top Level Field

To query documents where a field equals a specified value, use the filter document `{ field: value }`:

```
// Find all documents where status = "D":
db.inventory.find({ status: "D" })

// Equivalent SQL: SELECT * FROM inventory WHERE status = 'D'

// Find employees where city = "Delhi":
db.employees.find({ "address.city": "Delhi" })
```

9.3 Comparison Operators — Query Operators

Operator	Meaning & Example
<code>\$eq</code>	Equal to: <code>{ age: { \$eq: 20 } }</code> OR shorthand <code>{ age: 20 }</code>
<code>\$ne</code>	Not equal to: <code>{ status: { \$ne: 'inactive' } }</code>
<code>\$gt</code>	Greater than: <code>{ marks: { \$gt: 80 } }</code>
<code>\$gte</code>	Greater than or equal: <code>{ marks: { \$gte: 80 } }</code>
<code>\$lt</code>	Less than: <code>{ age: { \$lt: 30 } }</code>
<code>\$lte</code>	Less than or equal: <code>{ price: { \$lte: 500 } }</code>
<code>\$in</code>	Match any value in array: <code>{ status: { \$in: ['A','P'] } }</code>
<code>\$nin</code>	Not in array: <code>{ status: { \$nin: ['D','X'] } }</code>
<code>\$and</code>	Logical AND: <code>{ \$and: [{cond1}, {cond2}] }</code>
<code>\$or</code>	Logical OR: <code>{ \$or: [{cond1}, {cond2}] }</code>
<code>\$not</code>	Negation: <code>{ qty: { \$not: { \$gt: 100 } } }</code>
<code>\$exists</code>	Field exists: <code>{ phone: { \$exists: true } }</code>
<code>\$type</code>	Field type: <code>{ age: { \$type: 'int' } }</code>

\$regex

Pattern match: { name: { \$regex: "^Ak" } }

★ **Exam Tip:** *MST Q9: Query for marks between 70 and 90. Answer: a) { marks: { \$gt: 70, \$lt: 90 } }. This uses shorthand AND — multiple conditions on same field are implicitly ANDed.*

9.4 Query on Embedded / Nested Documents

To query fields inside embedded documents, use dot notation in quotes:

```
// Find employees where address.city = "Delhi":  
db.employees.find({ "address.city": "Delhi" })
```

```
// Find books where Book.writer = "Akash":  
db.books.find({ "Book.writer": "Akash" })
```

9.5 Query on Arrays

```
// Find students who have "Python" in skills array:  
db.students.find({ skills: "Python" })
```

```
// Find students with marks between 60 and 90:  
db.students.find({ marks: { $gt: 60, $lt: 90 } })
```

```
// Find students who know MongoDB AND marks between 60 and 90:  
db.students.find({  
  $and: [  
    { skills: "MongoDB" },  
    { marks: { $gt: 60, $lt: 90 } }  
  ]  
})
```

```
// Find students who know MongoDB (using $in on array):  
db.students.find({ skills: { $in: ["MongoDB"] } })
```

9.6 Projection — Select Specific Fields

Use a projection document to control which fields are returned. 1 = include, 0 = exclude. `_id` is included by default (set `_id: 0` to exclude it).

```
// Return only name and marks (exclude _id):  
db.students.find({}, { name: 1, marks: 1, _id: 0 })
```

```
// Return all fields except skills:  
db.students.find({}, { skills: 0 })
```

9.7 Sorting and Counting

```
// Sort by marks descending:  
db.students.find().sort({ marks: -1 })
```

```
// Sort ascending by name:  
db.students.find().sort({ name: 1 })
```

```
// Count matching documents:  
db.students.countDocuments({ skills: "Python" })
```

SECTION 10: Aggregation in MongoDB

Aggregation operations process multiple documents and return computed/summarised results. Unlike `find()` which just filters and returns documents, aggregation performs transformations, groupings, calculations, and joins. MongoDB aggregation uses a pipeline — a sequence of stages where each stage transforms the documents passing through it.

Aggregation Pipeline

A sequence of data processing stages. Documents enter the first stage, are transformed, then passed to the next stage. Each stage narrows, reshapes, or computes the data.

10.1 Common Aggregation Pipeline Stages

Stage	Description & Usage
<code>\$match</code>	Filters documents (equivalent to WHERE in SQL). Place early in pipeline for performance. Example: <code>{ \$match: { status: 'A' } }</code>
<code>\$group</code>	Groups documents by a key field; applies accumulators (<code>\$sum</code> , <code>\$avg</code> , <code>\$max</code> , <code>\$min</code> , <code>\$count</code>). Example: <code>{ \$group: { _id: '\$category', total: { \$sum: '\$price' } } }</code>
<code>\$sort</code>	Sorts documents in ascending (1) or descending (-1) order. Example: <code>{ \$sort: { total: -1 } }</code>
<code>\$project</code>	Reshapes documents; includes/excludes fields; adds computed fields. Example: <code>{ \$project: { name: 1, marks: 1, _id: 0 } }</code>
<code>\$limit</code>	Limits number of documents passed to next stage. Example: <code>{ \$limit: 5 }</code>
<code>\$skip</code>	Skips the first N documents. Example: <code>{ \$skip: 10 }</code>
<code>\$unwind</code>	Deconstructs an array field, creating one document per array element. Example: <code>{ \$unwind: '\$skills' }</code>
<code>\$lookup</code>	Performs a left outer join with another collection (like SQL JOIN). Example: join orders with products.
<code>\$count</code>	Counts documents and puts result in a field. Example: <code>{ \$count: 'totalStudents' }</code>
<code>\$addFields</code>	Adds new fields to documents without removing existing ones.

10.2 Aggregation — Accumulator Operators (used in `$group`)

Accumulator	Description
<code>\$sum</code>	Calculates sum of numeric values. <code>{ \$sum: '\$price' }</code> or <code>{ \$sum: 1 }</code> for count
<code>\$avg</code>	Calculates average of numeric values
<code>\$max</code>	Returns the maximum value in the group
<code>\$min</code>	Returns the minimum value in the group
<code>\$push</code>	Creates an array of values from the group

\$first	Returns the first value in the group (after sorting)
\$last	Returns the last value in the group (after sorting)
\$multiply	Multiplies fields together (used inside \$sum for revenue)

10.3 Example 1 — Total Revenue per Item (MST Q5)

Collection: orders { item, price, quantity }. Compute total revenue = price × quantity per item:

```
db.orders.aggregate([
  {
    $group: {
      _id: "$item",
      totalRevenue: {
        $sum: {
          $multiply: ["$price", "$quantity"]
        }
      }
    }
  }
])
```

10.4 Example 2 — \$match then \$group then \$sort

Find total amount per customer for orders with status='A', sorted by total descending:

```
db.orders.aggregate([
  { $match: { status: "A" } },           // Step 1: filter
  { $group: {                           // Step 2: group
    _id: "$cust_id",
    total: { $sum: "$amount" }
  } },
  { $sort: { total: -1 } }              // Step 3: sort
])
```

10.5 Example 3 — Unwind Array then Group

Find how many students have each skill:

```
db.students.aggregate([
  { $unwind: '$skills' },               // one doc per skill per student
  { $group: {
    _id: '$skills',
    count: { $sum: 1 }
  } },
  { $sort: { count: -1 } }
])
```

10.6 Example 4 — \$lookup (Join two collections)

Join orders collection with products collection on item field:

```
db.orders.aggregate([
  {
    $lookup: {
      from: "products",                // collection to join
      localField: "item",              // field in orders
      foreignField: "name",            // field in products
      as: "productDetails"            // output array field name
    }
  }
])
```



```
}  
])
```

10.7 Pipeline Performance Tips

- Always place \$match FIRST in the pipeline to filter early and reduce documents processed.
- Place \$sort before \$group when possible for better performance.
- Use \$project to reduce the number of fields early in the pipeline.
- Create indexes on fields used in \$match stages.

★ **Exam Tip:** Aggregation is a common 16-mark question. Know: \$match (filter), \$group (aggregate), \$sum (total), \$multiply (revenue = price×qty), \$sort, \$unwind (arrays), \$lookup (join).

SECTION 11: Indexing in MongoDB

Indexes are special data structures that store a small portion of the collection's data in an easy-to-traverse form. Without indexes, MongoDB must scan every document in a collection (collection scan / full scan) — this is like reading an entire book to find one word. Indexes are like the index at the back of a book — they let MongoDB jump directly to the right page.

Index	A sorted data structure (B-tree) that stores key values from a collection field and a pointer to the corresponding document. Allows MongoDB to find documents in $O(\log n)$ instead of $O(n)$.
--------------	--

11.1 Why Indexing is Needed

- Without index → MongoDB must examine EVERY document → $O(n)$ time → slow for large collections.
- With index → MongoDB traverses the B-tree → $O(\log n)$ time → dramatically faster.
- Indexes reduce CPU usage, disk I/O, and network overhead.
- Indexes are essential for queries on large datasets (millions of documents).

★ **Exam Tip:** *MST Q8: Which operation is most expensive without indexing? Answer: b) find() — performs a full collection scan $O(n)$ without an index.*

11.2 Creating Indexes — createIndex()

Syntax:

```
db.collection.createIndex({ field: direction }, options)
```

Direction: 1 = ascending, -1 = descending

```
// Single field ascending index:
db.orders.createIndex({ item: 1 })

// Single field descending index:
db.orders.createIndex({ price: -1 })

// Compound index (multiple fields):
db.orders.createIndex({ item: 1, price: -1 })

// Unique index (no duplicate values allowed):
db.students.createIndex({ email: 1 }, { unique: true })

// Text index (for full-text search):
db.articles.createIndex({ content: 'text' })
```

11.3 Types of Indexes in MongoDB

Index Type	Description & Use Case
Single Field Index	Index on one field. Most common type. <code>db.col.createIndex({ name: 1 })</code>
Compound Index	Index on multiple fields. Useful for multi-field queries. <code>db.col.createIndex({ a: 1, b: -1 })</code>
Multikey Index	Automatically created when indexing an array field. Indexes each element of the array.
Text Index	Enables full-text search on string content. <code>db.col.createIndex({ desc: 'text' })</code>

Hashed Index	Indexes the hash of a field value. Used for sharding. <code>db.col.createIndex({ _id: 'hashed' })</code>
Geospatial Index	For geospatial queries (location-based). 2d or 2dsphere indexes.
TTL Index	Time-To-Live index. Automatically deletes documents after a time. <code>db.logs.createIndex({ createdAt: 1 }, { expireAfterSeconds: 3600 })</code>
Unique Index	Ensures no duplicate values. <code>{ unique: true }</code> option.

11.4 Managing Indexes

```
// List all indexes on a collection:
db.students.getIndexes()

// Drop a specific index:
db.students.dropIndex({ name: 1 })

// Drop all indexes (except _id):
db.students.dropIndexes()

// Check how a query uses indexes (query plan):
db.students.find({ name: "Akash" }).explain("executionStats")
```

11.5 How Indexing Improves Performance

Without an index, MongoDB must perform a COLLSCAN (Collection Scan) — it reads every document to find matches. With an index, MongoDB performs an IXSCAN (Index Scan) — it traverses the B-tree directly to matching documents.

Without Index (COLLSCAN)	With Index (IXSCAN)
Reads ALL n documents	Traverses B-tree in $O(\log n)$ steps
Performance degrades linearly as collection grows	Performance stays fast even for millions of docs
High CPU and disk I/O	Minimal CPU and disk I/O
Suitable for small collections (<1000 docs)	Essential for large collections
No setup cost	Small storage cost + write overhead for each index

★ **Exam Tip:** Indexing improves *SELECT/find()* query performance. But indexes slow down *INSERT, UPDATE, DELETE* slightly because MongoDB must also update the index. Always index fields used frequently in *WHERE/find* conditions.

SECTION 12: Replica Sets & High Availability

A Replica Set is a group of MongoDB instances (mongod processes) that all maintain the same dataset. Replica sets provide data redundancy and high availability — the two most critical properties of any production database system.

Replica Set	A group of MongoDB servers that maintain identical copies of data. One primary receives writes; secondaries replicate data. If primary fails, a secondary is automatically elected as new primary.
--------------------	--

12.1 Components of a Replica Set

Member Type	Role & Description
Primary	Receives ALL write operations. Only one primary at a time. Logs all changes to the oplog (operations log). Clients read from primary by default.
Secondary	Replicates data from the primary by applying operations from the oplog. Can serve read requests (with read preference setting). Eligible to become primary if current primary fails.
Arbiter	Participates in elections to elect a new primary. Does NOT store data. Does NOT become primary. Used to break voting ties in small replica sets. Keeps costs low.

★ **Exam Tip:** *MST Q10: In a replica set, what happens if the primary node fails? Answer: c) One secondary becomes primary automatically (automatic failover via election).*

12.2 How Replica Sets Work

6. The PRIMARY node receives all write operations from clients.
7. All writes are recorded in the primary's OPLOG (operations log — a special capped collection).
8. SECONDARY nodes continuously poll the primary's oplog and apply the same operations to their own data (replication).
9. If the PRIMARY fails, the remaining eligible secondary members hold a VOTE (election).
10. The secondary with the most up-to-date oplog and highest priority wins the election and becomes the new PRIMARY.
11. Clients automatically reconnect to the new primary. Downtime is typically only a few seconds.
12. When the old primary comes back online, it rejoins as a SECONDARY (not primary).

12.3 Benefits of Replica Sets

- High Availability — Automatic failover. No manual intervention needed when primary fails.
- Data Redundancy — Multiple copies of data. No single point of failure.
- Read Scaling — Secondaries can serve read operations, distributing the read load.
- Disaster Recovery — If a data center fails, a replica in another DC takes over.
- Rolling Maintenance — Update/restart one member at a time without downtime.

12.4 Replica Set Commands

```
// Initialize a replica set:  
rs.initiate()
```

```
// Check the status of a replica set:
```

```
rs.status()

// Add a secondary member:
rs.add("mongodb1.example.com:27017")

// Add an arbiter:
rs.addArb("mongodb2.example.com:27017")

// Check replica set configuration:
rs.conf()

// Step down the primary (force election):
rs.stepDown()
```

12.5 Minimum Replica Set Configuration

A recommended minimum replica set has 3 members to ensure proper elections:

- Option A: 3 data-bearing members (1 primary + 2 secondaries). Tolerates 1 failure.
- Option B: 2 data-bearing members (1 primary + 1 secondary) + 1 arbiter. Tolerates 1 failure.

 **Note:** *An odd number of voting members is recommended to avoid election ties. Maximum of 50 members in a replica set, but only 7 can be voting members.*

★ **Exam Tip:** *Replica Set vs Sharding: Replica Sets = same data on multiple nodes (availability). Sharding = different data on different nodes (horizontal scaling). Both can be used together.*

END SEMESTER — 16 MARK QUESTION ANSWERS (Complete Practice)

16M-Q1: Explain all Data Types in MongoDB BSON with examples. (16 Marks)

Introduction (2 marks): In MongoDB, documents are stored in BSON (Binary Encoded Serialization Of JSON). BSON supports additional data types beyond JSON. MongoDB provides 18 data types accessible in the Mongo Shell.

Data Type	Description (2-3 lines)	Code Example
1. String	Most common. Must be valid UTF-8. Drivers convert language strings to UTF-8 during serialization.	<code>db.student.insertOne({ name: "Akash" })</code>
2. Integer	Stores whole numbers. Two forms: 32-bit signed and 64-bit signed (NumberLong).	<code>db.student.insertOne({ age: 18 })</code>
3. Double	Stores floating-point (decimal) values. IEEE 754 64-bit format.	<code>db.student.insertOne({ marks: 546.63 })</code>
4. Boolean	Stores true or false. Used for binary states (pass/fail, active/inactive).	<code>db.student.insertOne({ pass: true })</code>
5. Null	Stores null value. Field exists but has no value.	<code>db.student.insertOne({ MobNo: null })</code>
6. Array	Set of values in square brackets []. Same or different types allowed.	<code>db.student.insertOne({ skills: ["C","Java"] })</code>
7. Object	Embedded/nested document. A document inside another document.	<code>db.book.insertOne({ Book: { name: "C in depth" } })</code>
8. ObjectId	12-byte auto-generated unique <code>_id</code> . 4(ts)+3(machine)+2(process)+3(counter).	<code>db.book.find().pretty() → "_id": ObjectId(...)</code>
9. Undefined	Deprecated. Stores undefined value. Different from null.	<code>db.p.insertOne({ duration: undefined })</code>
10. Binary Data	Stores raw binary data — images, encrypted content, files.	<code>db.p.insertOne({ binaryValue: "11001100" })</code>
11. Date	64-bit milliseconds since Unix epoch. UTC. <code>Date()</code> =string, <code>new Date()</code> =object.	<code>var d = new Date(); db.d.insertOne({ date: d })</code>
12. Min/Max Key	Internal BSON types. <code>MinKey</code> = lowest BSON value. <code>MaxKey</code> = highest.	<code>db.m.insertOne({ min: MinKey, max: MaxKey })</code>
13. Symbol	Like String but deprecated. Shell auto-converts to string.	<code>var s="nik432#"; db.symb1.insert({ symbol: s })</code>
14. Regex	Stores regular expressions for pattern-based queries.	<code>db.exp.insertOne({ exp: new RegExp("^gfg") })</code>
15. JavaScript	Stores JS code without scope in the document.	<code>db.js.insertOne({ fn: function(){} })</code>
16. JS + Scope	Stores JS code with variable scope mapping.	Code + scope document together
17. Timestamp	64-bit. Used in oplog/replication to track data modifications.	<code>db.time.insertOne({ ts: new Timestamp() })</code>
18. Decimal128	128-bit high-precision decimal. Used for financial calculations.	<code>db.d.insertOne({ bal: NumberDecimal("1000.55") })</code>

★ **Exam Tip:** *ObjectId composition is a direct 1-mark MCQ: $4+3+2+3 = 12$ bytes total. Also: Decimal128 is more precise than Double; used for money.*

16M-Q2: Explain Inserting Documents — insertOne, insertMany, insert, save — with all examples. (16 Marks)

Introduction (2 marks):

MongoDB stores data as BSON documents in collections. The Shell provides several methods for inserting documents. If a document lacks an `_id` field, MongoDB automatically adds one (ObjectId).

1. insertOne() — Single Document Insert (4 marks):

Inserts exactly one document. Returns `{ acknowledged: true, insertedId: ObjectId }`. Creates `_id` automatically if not provided.

```
db.EmpDetails.insertOne({
  firstname: "Radhika", lastname: "Sharma",
  DateOfBirth: "1995-09-26",
  email: "radhika.123@gmail.com",
  phone: "9848022338"
})
```

2. insertMany() — Multiple Documents (4 marks):

Inserts an array of documents in one operation. Much faster than calling `insertOne()` in a loop. By default unordered — continues on error.

```
db.movies.insertMany([
  { title: "Jurassic World: Fallen Kingdom", genres: ["Action","Sci-Fi"], year: 2016, type: "movie" },
  { title: "Tag", genres: ["Comedy","Action"], year: 2018, type: "movie" }
])
```

3. insert() Method — Legacy (2 marks):

Can insert single or multiple documents. Deprecated in new versions but still valid in exams. Returns `WriteResult` for single, `BulkWriteResult` for array.

```
db.users.insert({
  title: "MongoDB overview",
  tags: ["mongodb","database","NoSQL"],
  likes: 100
})
```

4. save() Method — Upsert Behaviour (4 marks):

`save()` has two behaviours: (a) No `_id` → acts as INSERT. (b) Has `_id` → acts as REPLACE (update with `upsert:true`). Entire document is replaced — not just updated fields.

```
// Behaviour A: INSERT (no _id field):
db.products.save({ item: 'Book', qty: 40 })

// Behaviour B: REPLACE (has _id field):
db.product.save({ _id: 100, item: 'water', qty: 30 })
// If _id:100 exists → replace entire doc.
// If not exists → insert as new doc.
```

Method	Key Behaviour
<code>insertOne(doc)</code>	Insert 1 doc. Returns <code>insertedId</code> .
<code>insertMany([docs])</code>	Insert N docs in 1 operation. Returns <code>insertedIds</code> array.
<code>insert(doc/array)</code>	Legacy. Single or array. Returns <code>WriteResult/BulkWriteResult</code> .
<code>save(doc)</code>	No <code>_id</code> : INSERT. Has <code>_id</code> : REPLACE entire document (<code>upsert</code>).

16M-Q3: Explain Querying in MongoDB — All Documents, Top Level Field, Operators, Embedded Fields, Arrays, \$and/\$or. (16 Marks)

Introduction (1 mark):

MongoDB uses the find() method for querying. Queries are expressed as BSON documents. MongoDB supports rich comparison operators for complex queries.

Syntax:

```
db.collection.find(filter, projection)
```

1. Query All Documents (2 marks):

```
db.inventory.find({}) // returns ALL documents
db.inventory.find({}).pretty() // formatted output
// SQL equivalent: SELECT * FROM inventory
```

2. Query by Top Level Field (2 marks):

```
db.inventory.find({ status: "D" }) // exact match
db.students.find({ marks: { $gt: 80 } }) // greater than 80
db.students.find({ marks: { $gt: 70, $lt: 90 } }) // range 70-90
```

3. Comparison Operators (3 marks):

Operator	Example Query
\$gt (greater than)	db.students.find({ marks: { \$gt: 80 } })
\$lt (less than)	db.students.find({ marks: { \$lt: 40 } })
\$gte (>=)	db.products.find({ price: { \$gte: 500 } })
\$lte (<=)	db.products.find({ qty: { \$lte: 10 } })
\$in (any of)	db.orders.find({ status: { \$in: ["A","P"] } })
\$ne (not equal)	db.inventory.find({ type: { \$ne: "food" } })

4. Query on Embedded Fields — Dot Notation (2 marks):

```
// Find where address.city = "Delhi":
db.employees.find({ "address.city": "Delhi" })

// Find where Book.writer = "Akash":
db.books.find({ "Book.writer": "Akash" })
```

5. Query on Arrays (2 marks):

```
// Students with "Python" in skills array:
db.students.find({ skills: "Python" })

// Students who have BOTH Python and MongoDB:
db.students.find({ skills: { $all: ["Python", "MongoDB"] } })
```

6. Logical Operators — \$and, \$or (2 marks):

```
// $and — combine conditions (explicit):
db.students.find({
  $and: [
    { skills: "MongoDB" },
    { marks: { $gt: 60, $lt: 90 } }
  ]
})
```

```
// $and shorthand (implicit — just list conditions):
db.students.find({ skills: "MongoDB", marks: { $gt: 60 } })

// $or — at least one must match:
db.students.find({
  $or: [
    { marks: { $gt: 90 } },
    { skills: "Python" }
  ]
})
```

7. Limit, Sort, Skip — Pagination (2 marks):

```
// Sort by marks desc, limit to 5:
db.students.find().sort({ marks: -1 }).limit(5)

// Skip first 10, get next 10 (page 2):
db.students.find().sort({ name: 1 }).skip(10).limit(10)
```

16M-Q4: Explain Aggregation Pipeline in MongoDB with all stages and examples. (16 Marks)

Introduction (2 marks):

Aggregation in MongoDB is the process of transforming and computing data from multiple documents. The aggregation pipeline is a series of stages where documents flow through and are processed at each stage. Unlike `find()`, aggregation can compute totals, averages, groups, joins, and reshaping.

Syntax:

```
db.collection.aggregate([
  { stage1 },
  { stage2 },
  { stage3 }
])
```

\$match — Filter Stage (2 marks):

Filters documents passing through the pipeline. Equivalent to WHERE in SQL. Should be placed FIRST for performance.

```
db.orders.aggregate([ { $match: { status: "A" } } ])
```

\$group — Group & Aggregate (3 marks):

Groups documents by a field (`_id`) and applies accumulator expressions. Most important stage.

```
db.orders.aggregate([
  {
    $group: {
      _id: "$item",          // group by item
      totalRevenue: {
        $sum: {
          $multiply: ["$price", "$quantity"] // revenue = price × qty
        }
      },
      avgPrice: { $avg: '$price' }, // average price per item
      count: { $sum: 1 }           // count orders
    }
  }
])
```

\$sort — Sort Stage (1 mark):

```
db.orders.aggregate([ { $sort: { totalRevenue: -1 } } ]) // desc
```

\$project — Reshape Stage (2 marks):

Include/exclude fields, rename fields, create computed fields.

```
db.students.aggregate([
  { $project: { name: 1, marks: 1, _id: 0,
    grade: { $cond: { if: { $gte: ['$marks', 75] }, then: 'Pass', else: 'Fail' } } } }
])
```

\$unwind — Array Deconstruct Stage (2 marks):

Turns one document with an array into many documents (one per array element).

```
// Before: { name:'Akash', skills:['C','Java','Python'] }
// After $unwind: 3 separate documents, one per skill
db.students.aggregate([ { $unwind: '$skills' } ])
```

\$lookup — Join Stage (2 marks):

```
db.orders.aggregate([
  { $lookup: {
    from: "products",
    localField: "item",
    foreignField: "name",
    as: "productInfo"
  } }
])
```

Complete Pipeline Example (2 marks):

```
// Revenue by item for 'Electronics', sorted high to low:
db.orders.aggregate([
  { $match: { category: "Electronics" } }, // filter
  { $group: {
    _id: "$item",
    revenue: { $sum: { $multiply: ['$price', '$quantity'] } }
  } },
  { $sort: { revenue: -1 } }, // sort desc
  { $limit: 5 } // top 5
])
```

16M-Q5: Explain Updating Documents — updateOne, updateMany, Update Embedded Field, Replace a Document, replaceOne. (16 Marks)

Introduction (1 mark):

MongoDB provides update operators such as \$set to modify field values without replacing the entire document. The three main update methods are updateOne(), updateMany(), and replaceOne(). Update operators allow targeted modifications.

Key Update Operators (2 marks):

Operator	Description
\$set	Sets field value. Creates field if doesn't exist.
\$unset	Removes the specified field from document.
\$inc	Increments field by a given amount.
\$mul	Multiplies field by a given number. \$mul: {price: 1.10} = 10% increase.
\$push	Appends a value to an array.
\$pull	Removes matching values from array.
\$currentDate	Sets field to current date/time.

1. updateOne() — Update Single Document (3 marks):

Updates the FIRST document matching the filter. Only specified fields are changed; all others remain.

```
await db.collection('inventory').updateOne(
  { item: 'Paper' },
  {
    $set: { 'size.uom': 'cm', status: 'P' },
    $currentDate: { lastModified: true }
  }
);
```

2. updateMany() — Update Multiple Documents (3 marks):

Updates ALL documents matching the filter. Use with care — can affect many records.

```
await db.collection('inventory').updateMany(
  { qty: { $lt: 50 } },
  {
    $set: { 'size.uom': 'in', status: 'P' },
    $currentDate: { lastModified: true }
  }
);

// Increase all Electronics prices by 10%:
db.products.updateMany(
  { category: "Electronics" },
  { $mul: { price: 1.10 } }
);
```

3. Update an Embedded Field (3 marks):

Use dot notation in \$set to update nested/embedded document fields without affecting parent or sibling fields.

```
// Update nested field address.city:
db.employees.updateMany(
  { "address.city": "Delhi" },
```

```

    { $set: { "address.city": "Mumbai" } }
  )

// Update housing street field:
db.housing.updateOne(
  { "_id": "E123" },
  { $set: { "address.$street": "elm mews" } }
)

```

4. replaceOne() — Replace Entire Document (4 marks):

replaceOne() replaces the ENTIRE document (except _id). All old fields are removed. New document's fields take their place. The replacement document MUST NOT use update operators.

```

// Replace employee Tamanna completely:
db.employees.replaceOne(
  { name: "Tamanna" }, // filter
  {                     // new replacement document
    name: "Sanjeevni",
    salary: 60000,
    address: {
      city: "Pune",
      pincode: 411001
    }
  }
)

```

replaceOne()	updateOne() + \$set
Replaces ENTIRE document	Updates ONLY specified fields
All old fields REMOVED	All old fields KEPT
No \$set operator used	Must use \$set or other operators
_id is preserved	_id is preserved
Like overwriting a record	Like editing specific columns

★ **Exam Tip:** KEY: replaceOne() FALSE about keeping existing fields. \$set on non-existing field CREATES it. updateMany updates ALL matching. Always use dot notation ('address.city') for nested field updates.

16M-Q6: Explain Indexing & Replica Sets in MongoDB. How do they improve performance and availability? (16 Marks)

PART A — INDEXING (8 marks):

Introduction (1 mark): Indexes are special B-tree data structures that store a small portion of the collection's data in easy-to-traverse form. Without indexes, MongoDB performs a collection scan ($O(n)$). With indexes, queries run in $O(\log n)$.

Creating an Index (2 marks):

```
db.orders.createIndex({ item: 1 }) // ascending
db.orders.createIndex({ price: -1 }) // descending
db.orders.createIndex({ item: 1, price: -1 }) // compound
db.students.createIndex({ email: 1 }, { unique: true }) // unique
```

Types of Indexes (2 marks):

Index Type	Description
Single Field	Index on one field. Most common.
Compound	Index on multiple fields. Supports multi-field queries.
Multikey	Auto-created for array fields. Indexes each element.
Text	Full-text search. { field: 'text' }
TTL	Auto-delete docs after time: { expireAfterSeconds: N }
Unique	Prevents duplicate values: { unique: true }

Why Indexing Improves Performance (3 marks):

Without index (COLLSCAN): MongoDB reads EVERY document — $O(n)$. 1 million docs = 1 million reads.

With index (IXSCAN): MongoDB traverses the B-tree — $O(\log n)$. 1 million docs $\approx$ 20 steps.

- Reduces query time dramatically for large datasets.
- Reduces CPU usage and disk I/O.
- Essential for production systems.
- Small cost: index takes storage + write operations update the index.

```
// Check if query uses index:
db.students.find({ name: "Akash" }).explain("executionStats")
// Look for: "IXSCAN" (with index) vs "COLLSCAN" (without index)
```

PART B — REPLICA SETS (8 marks):

Introduction (1 mark): A Replica Set is a group of MongoDB instances maintaining identical datasets. It provides data redundancy, high availability, and automatic failover.

Components (2 marks):

Member	Role
Primary	Receives ALL writes. Logs to oplog. One at a time.
Secondary	Replicates from primary's oplog. Can serve reads. Can become primary.

Arbiter	Votes in elections. No data. Breaks ties in even-numbered sets.
---------	---

How It Works — Replication Flow (3 marks):

13. Client sends write to PRIMARY.
14. Primary records write in OPLOG (capped collection — operations log).
15. SECONDARIES continuously poll oplog and apply same operations.
16. PRIMARY fails → remaining members VOTE (election).
17. Secondary with most up-to-date oplog wins election → becomes new PRIMARY.
18. Clients auto-reconnect to new primary. Downtime: seconds.
19. Old primary recovers → rejoins as SECONDARY.

Commands (1 mark):

```
rs.initiate()    // initialize replica set
rs.status()     // view member statuses
rs.add("host:port") // add a secondary
rs.stepDown()   // force primary to step down
```

Benefits (1 mark):

- High Availability: automatic failover, no manual intervention.
- Data Redundancy: multiple data copies, no single point of failure.
- Read Scaling: secondaries can serve reads, distributing load.

★ **Exam Tip:** *Replica Set ensures HIGH AVAILABILITY. Sharding ensures HORIZONTAL SCALABILITY. Both can coexist. Replica set election is automatic — no manual restart needed.*

MASTER QUICK REVISION TABLE — UNIT 3 (All Key Facts)

Topic / Question	Answer / Key Fact
ObjectId size	12 bytes = 4(timestamp)+3(machine)+2(process)+3(counter)
BSON stands for	Binary Encoded Serialization Of JSON
insertMany() on fail	Continues by default (unordered). Ordered insertMany stops on error.
insertOne() vs insert()	insertOne is preferred; insert() is deprecated but still tested.
save() with _id field	Performs UPDATE with upsert:true — replaces entire document
save() without _id field	Performs INSERT — new document created
deleteMany({}) empty filter	DELETES ALL documents in collection (collection remains)
deleteOne() behaviour	Deletes FIRST matching document only
\$set on non-existing field	CREATES the field automatically
\$mul: { price: 1.10 }	Multiplies price by 1.10 = 10% increase
updateMany vs updateOne	updateMany: ALL matching docs. updateOne: FIRST matching only.
replaceOne() behaviour	REMOVES all old fields and replaces with new document entirely
replaceOne vs updateOne+\$set	replaceOne: full replacement. updateOne+\$set: partial update.
Embedded field update syntax	Use dot notation: { \$set: { "address.city": "Mumbai" } }
\$push operator	Adds an element to an array field
Marks between 70 and 90	{ marks: { \$gt: 70, \$lt: 90 } }
Query all documents	db.collection.find({})
SQL SELECT * equivalent	db.inventory.find({})
Batch insert types	Unordered (continues on error) + Ordered (stops on error)
bulk.execute()	Must be called to actually perform bulk operations
Most expensive without index	find() — full collection scan O(n)
createIndex() syntax	db.collection.createIndex({ field: 1 }) // 1=asc, -1=desc
Index types	Single, Compound, Multikey, Text, TTL, Unique, Hashed, Geospatial

COLLSCAN vs IXSCAN	COLLSCAN = no index (slow). IXSCAN = uses index (fast).
Replica set primary fails	Secondary becomes primary AUTOMATICALLY via election
Replica set Arbiter role	Votes in elections ONLY. No data stored. Breaks ties.
Oplog purpose	Records all primary write operations for secondaries to replicate
Aggregation \$group	Groups docs by _id field, applies accumulators (\$sum, \$avg etc.)
Revenue formula in aggregation	\$sum: { \$multiply: ['\$price', '\$quantity'] }
\$unwind stage	Deconstructs array into one document per array element
\$lookup stage	Performs LEFT OUTER JOIN with another collection
\$match best practice	Place \$match FIRST in pipeline for performance
Limit syntax	db.collection.find().limit(n) OR { \$limit: n } in aggregate
Max document size	16 MB (MongoDB hard limit)
Max indexes per collection	64 indexes
Date() vs new Date()	Date() = string. new Date() = Date object (ISODate).
Difference: Null vs Undefined	null = field exists, no value. undefined = deprecated, unassigned.
Replica set minimum	3 members: 1 Primary + 2 Secondaries (or 2 data + 1 Arbiter)
rs.status() command	Shows status of all replica set members (primary/secondary/etc.)

MST-II PART A — MCQ ANSWERS (All 10 Questions)

Q.No	Question Summary	Correct Answer + Explanation
Q1	Which statement about insertMany() is correct?	c) Can insert multiple documents in a single operation. Does NOT stop on failure by default.
Q2	Which operator updates a nested (embedded) field?	c) Dot notation with \$set. Use 'field.subfield' inside { \$set: { } }.
Q3	What happens if \$set is used on a non-existing field?	c) Field is CREATED automatically by \$set.
Q4	True/False: MongoDB strictly enforces schema?	FALSE. MongoDB is schema-less (flexible schema). Documents in same collection can have different fields.
Q5	True/False: replaceOne() keeps existing fields not in new doc?	FALSE. replaceOne() REMOVES all old fields. Only new document's fields remain (except _id).
Q6	A field inside another document is called a ___ field.	EMBEDDED (or nested) field.
Q7	Operator used to add elements to an array is ___.	\$push. Syntax: { \$push: { arrayField: newValue } }
Q8	Most expensive operation without indexing?	b) find() — performs full collection scan O(n). No index = read every document.
Q9	Query for marks between 70 and 90?	a) { marks: { \$gt: 70, \$lt: 90 } } — implicit AND on same field.
Q10	In replica set, what happens if primary node fails?	c) One secondary becomes primary automatically via election process.

MST-II PART B — 5 MARK QUESTIONS WITH COMPLETE ANSWERS

Part B - Q1: Collection students {name, age, course, marks, skills[]} — (a) Insert 3 docs (b) Find Python+marks>80 (c) Update to add status Topper

(a) Insert 3 student documents with different skills arrays (1.5 marks):

```
db.students.insertMany([
  {
    name: "Tamanna", age: 20, course: "BTechCSE",
    marks: 85, skills: ["Python", "MongoDB"]
  },
  {
    name: "Anjali", age: 21, course: "BTechAS",
    marks: 78, skills: ["Java", "C++"]
  },
  {
    name: "Karan", age: 20, course: "BTechIT",
    marks: 92, skills: ["Python", "Data Science"]
  }
])
```

(b) Find students with Python skill AND marks > 80 (2 marks):

```
db.students.find({
  skills: "Python",
  marks: { $gt: 80 }
})
```

(c) Update all such students by adding field status: 'Topper' (1.5 marks):

```
db.students.updateMany(
  {
    skills: "Python",
    marks: { $gt: 80 }
  },
  { $set: { status: "Topper" } }
)
```

Part B - Q2: Collection products {name, price, category, stock} — (a) 10% price increase Electronics (b) Delete stock=0 (c) Find Electronics price>20000

(a) Increase price of all Electronics products by 10% (2 marks):

```
db.products.updateMany(
  { category: "Electronics" },
  { $mul: { price: 1.10 } } // multiply by 1.10 = 10% increase
)
```

(b) Delete products where stock = 0 (1 mark):

```
db.products.deleteMany({ stock: 0 })
```

(c) Find products where price > 20000 and category = 'Electronics' (2 marks):

```
db.products.find({
  category: "Electronics",
  price: { $gt: 20000 }
})
```

Part B - Q3: Collection employees {name, salary, address:{city,pincode}} — (a) Find Delhi (b) Update city to Mumbai embedded (c) Replace one employee

(a) Find employees where city = 'Delhi' (1.5 marks):

```
db.employees.find({
  "address.city": "Delhi"
})
```

(b) Update city to 'Mumbai' using embedded field dot notation (1.5 marks):

```
db.employees.updateMany(
  { "address.city": "Delhi" },
  {
    $set: {
      "address.city": "Mumbai"
    }
  }
)
```

(c) Replace one employee document completely with new data (2 marks):

```
db.employees.replaceOne(
  { name: "Tamanna" }, // find Tamanna
  { // replace ENTIRELY
    name: "Sanjeevni",
    salary: 60000,
    address: {
      city: "Pune",
      pincode: 411001
    }
  }
)
```

```
}  
)
```

Part B - Q4: Collection students {name, skills:[], marks} — (a) Find who know MongoDB (b) Marks 60-90 (c) Combine both with \$and

(a) Find students who know 'MongoDB' (1.5 marks):

```
db.students.find({ skills: "MongoDB" })
```

(b) Find students with marks between 60 and 90 (1.5 marks):

```
db.students.find(  
  marks: {  
    $gt: 60,  
    $lt: 90  
  }  
)
```

(c) Combine both conditions using \$and (2 marks):

```
db.students.find(  
  $and: [  
    { skills: "MongoDB" },  
    { marks: { $gt: 60, $lt: 90 } }  
  ]  
)
```

Part B - Q5: Collection orders {item, price, quantity} — (a) Aggregation for total revenue per item (b) Create index on item (c) Indexing performance (d) Replica sets

(a) Aggregation to compute total revenue = price × quantity per item (2 marks):

```
db.orders.aggregate([  
  {  
    $group: {  
      _id: "$item",  
      totalRevenue: {  
        $sum: {  
          $multiply: ["$price", "$quantity"]  
        }  
      }  
    }  
  },  
  { $sort: { totalRevenue: -1 } } // highest revenue first  
)
```

(b) Create an index on the item field (1 mark):

```
db.orders.createIndex({ item: 1 })  
// 1 = ascending index on 'item' field
```

(c) Why indexing improves performance (1 mark):

Without index: MongoDB scans ALL documents ($O(n)$ — collection scan). With index: MongoDB uses B-tree structure to locate documents in $O(\log n)$. For 1 million documents, this means ~1,000,000 reads vs ~20 steps. Indexing drastically reduces query time, CPU usage, and disk I/O. Indexes are stored separately and sorted — making lookups extremely fast.

(d) How replica sets ensure high availability (1 mark):

A Replica Set maintains multiple copies of data across servers. One PRIMARY receives all writes; SECONDARIES replicate via oplog. If the primary fails, an automatic election selects a new primary (in seconds) without any manual intervention. This ensures continuous availability even during hardware failures, network issues, or maintenance. Clients automatically reconnect to the new primary.

All the Best for Your End Semester Examinations! 🎓

Unit 3 — Basic MongoDB Operations | SRM University Delhi-NCR | Subject: 23CAF3010 NOSQL & MongoDB

*Covers: All 18 Data Types | insertOne/Many/insert/save | Batch Insert | deleteOne/Many | updateOne/Many | replaceOne | Embedded
Field Update | Top Level Field | Commands | limit() | Query All / Top Level | Aggregation
(\$match,\$group,\$sort,\$project,\$unwind,\$lookup) | Indexing | Replica Sets | MST-II Full Solutions*